A shortcut to get this view, since it is stored at resources/views/greeting.blade.php is

```
Route::get('/', function () {
    return view('greeting', ['name' => 'James']); });
```

the first argument passed to the view helper corresponds to the name of the view file in the resources/views directory. The second argument is an array of data that should be made available to the view. In this case, we are passing the name variable, which is displayed in the view using Blade syntax.

Sidebar: Blade is Laravel's simple and powerful templating engine. All Blade views are compiled into plain PHP code and cached until they are modified. Blade view files use the .blade.php file extension and are typically stored in the resources/views directory.

As we saw, the Hello World view already exists and we could simply reference that to get the view. If you wish to see if a view already exists, you can check it using the `exists` method and if it does exist, the return value will be `true`.

```
use Illuminate\Support\Facades\View;
if (View::exists('emails.customer')) {
    // }
```

A view composer is a helpful tool that can help you organise the logic for a particular if it is to be rendered everytime. It binds a data set to the view so it provides a nifty shortcut to code your view logic. Remember there is no default directory for view composers but you can create one and organise it as you wish. A view creator is similar to a composer with the difference being that they are called up instantly rather than waiting for the rendering process.

Your application might have user customizable displays within defined parameters. This means that there will be an array of views for that particular page. In this case you will need to set a 'first' view, the view that the user sees before any possible customization. This can be done using the `first` method

```
return view()->first(['custom.admin', 'admin'], $data);
```

You can also use the given facade, kind of a preset code provided by Laravel which provide access to almost all of it's features. It provides the benefit of a terse, expressive syntax while maintaining more testability and flexibility than traditional static methods.